

Mid Term Report

GROUP 12

Deep Neural Network Compression & CV

Made By:

Ashmil Saxena (Team Leader)

Team Members:

Aman Agrawal,
Ayush Kumar,
Harsh Verma,
Kushi Sharma,
Laukik Krishna Joshi,
Mayank Kumar,
Mayank Verma,
Sohan Kolhe
Yashvardhan Singh Chauhan

Part I: Fundamentals of Image Representation

1 Introduction to Digital Image Representation

In Computer Vision (CV), an image is treated not merely as visual data but as a mathematical matrix. While the human eye perceives a scene, the computer interprets a grid of numerical values.

1.1 The Image Matrix

A digital color image is structurally a multidimensional matrix with dimensions $H \times W \times 3$:

- H : Height in pixels.
- W : Width in pixels.
- 3: The three color channels, typically Red, Green, and Blue (RGB).

Each element in this grid is a pixel containing a tuple (R, G, B) . These values are typically stored as 8-bit unsigned integers (`uint8`) ranging from 0 to 255. However, for mathematical operations such as Fourier Transforms, these are often converted to floating-point numbers (`float32`) [2].

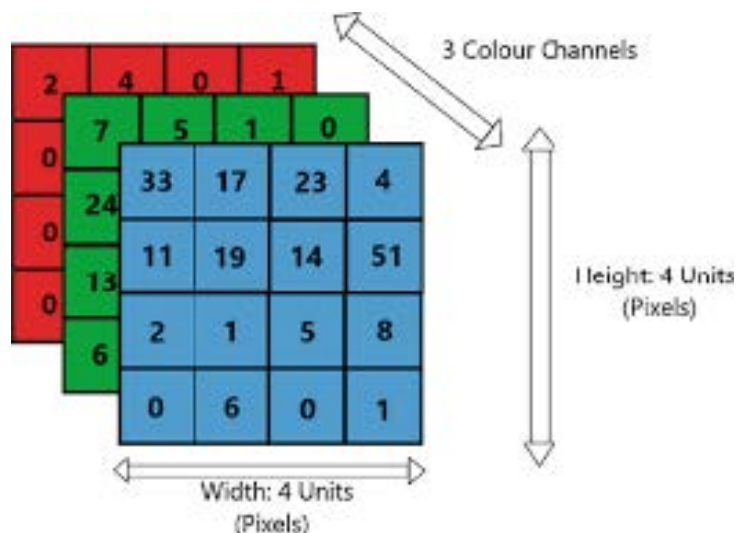


Figure 1: Visual representation of an image as three stacked matrices (Red, Green, Blue).

1.2 Loading Images

When using libraries like OpenCV and Matplotlib, channel ordering is a common pitfall. OpenCV loads images in **BGR** format by default, whereas standard display libraries expect **RGB**. A conversion is required for correct color visualization:

```
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

2 Grayscale Conversion

Grayscale images are preferred in many CV pipelines—including edge detection and FFT analysis—because they simplify the data from three channels to one, reducing computational load.

There are two primary methods to convert RGB to Grayscale (I_{GRAY}):

1. Average Method:

$$I_{GRAY} = \frac{R + G + B}{3} \quad (1)$$

2. **Weighted (Luminosity) Method:** This aligns with human perception, where green is perceived as brighter than red or blue.

$$I_{GRAY} = 0.299R + 0.587G + 0.114B \quad (2)$$

3 The Frequency Domain

The frequency domain breaks an image down into a sum of sinusoids of varying frequencies, rather than pixels at specific coordinates. This is achieved via the Fourier Transform.

3.1 FFT vs. DFT

- **DFT (Discrete Fourier Transform):** The mathematical definition. It has a high time complexity of $O(N^2)$, making it slow for large images.
- **FFT (Fast Fourier Transform):** An optimized algorithm with complexity $O(N \log N)$, enabling efficient processing.

3.2 Interpreting Frequency

- **Low Frequencies:** Correspond to smooth regions, gradients, background, and soft transitions.
- **High Frequencies:** Correspond to edges, fine textures, noise, and sharp details.

4 Magnitude and Phase Spectra

The output of an FFT is complex-valued, consisting of:

- **Magnitude Spectrum:** Indicates *how much* of a frequency is present (energy). It is typically visualized on a logarithmic scale:

$$Magnitude = \log(|F_{(u,v)}| + 1) \quad (3)$$

- **Phase Spectrum:** Indicates *where* the frequencies are located. Phase preserves the structural information (edges and shapes) of the image.

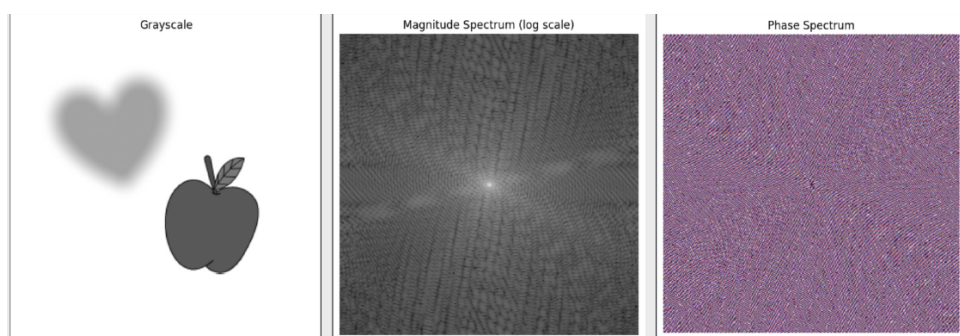


Figure 2: Comparison of Grayscale input, Magnitude Spectrum (Energy), and Phase Spectrum (Structure).

5 Filtering with Masks

We can manipulate images by applying masks in the frequency domain.

5.1 Low Pass Filter (LPF)

An LPF allows low frequencies to pass while blocking high frequencies.

- **Mask:** White circle (1) in the center, black (0) outside.
- **Result:** Blurring, smoothing, and noise removal.

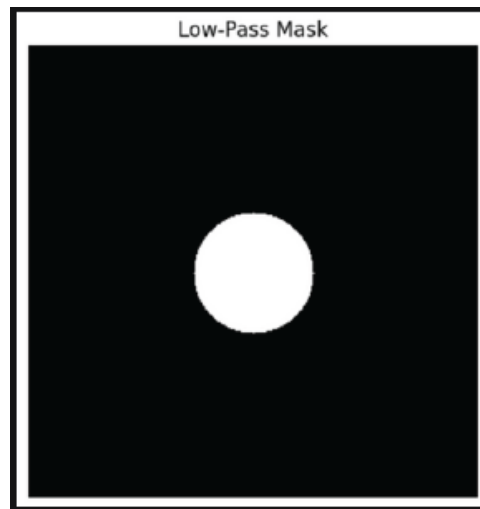


Figure 3: Low Pass Filter Mask.

5.2 High Pass Filter (HPF)

An HPF allows high frequencies to pass while blocking low frequencies.

- **Mask:** Black circle (0) in the center, white (1) outside.
- **Result:** Edge enhancement, sharpening, and detail extraction.

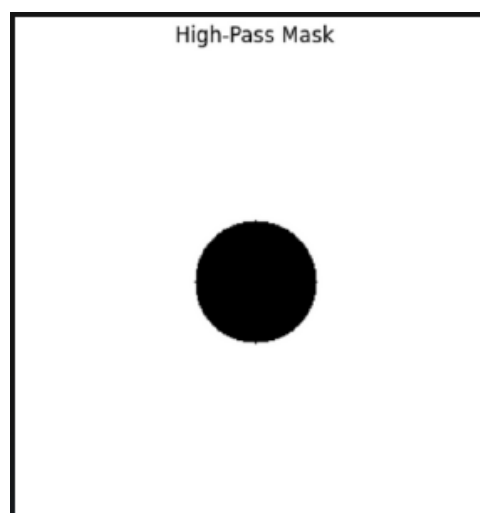


Figure 4: High Pass Filter Mask.

Part II: Image Analysis and Enhancement

6 Histograms

A histogram is a graphical representation of the distribution of pixel intensities in an image. It plots the number of pixels for each intensity value ranging from 0 to 255 [5].

6.1 Grayscale Histograms

In a grayscale histogram:

- **X-axis:** Represents intensity (0 = Black, 255 = White).
- **Y-axis:** Represents the pixel count.

The shape of the histogram reveals the exposure of the image [6]:

- Leaning Left → Dark Image.
- Centered → Balanced Exposure.
- Leaning Right → Bright Image.
- Wide Distribution → High Contrast.

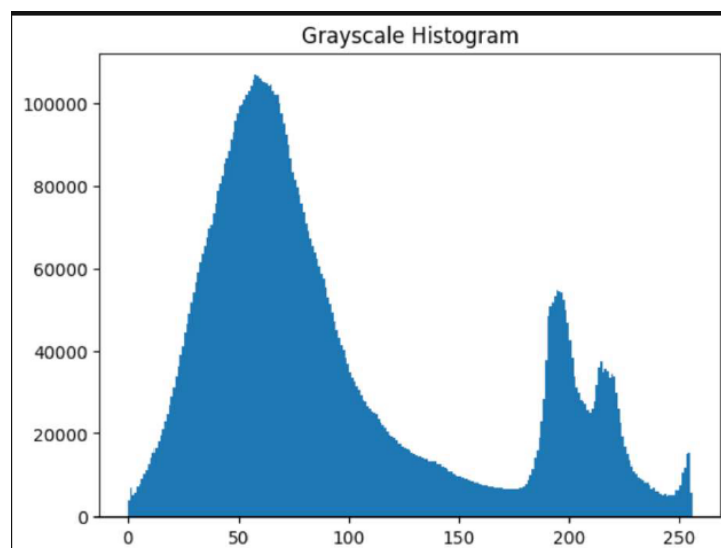


Figure 5: Grayscale Histogram showing pixel intensity distribution.

6.2 RGB Histograms

An RGB histogram plots separate distributions for the Red, Green, and Blue channels. This allows for the analysis of color balance and temperature (e.g., high Red values indicate a "warm" image).

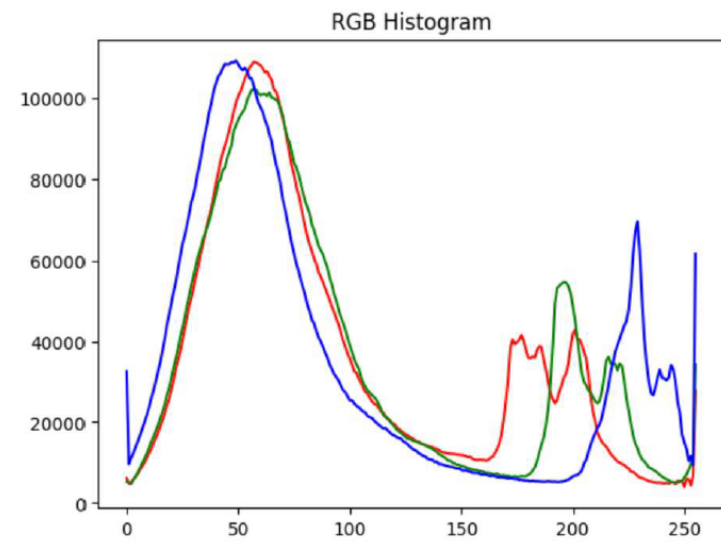


Figure 6: RGB Histogram showing individual color channel distributions.

7 Color Spaces: HSV and HSL

While RGB is standard for digital displays, it is not intuitive for human perception. Color spaces like HSV (Hue, Saturation, Value) align better with how we describe color.

7.1 HSV Components

- **Hue (H):** The "pure" color, represented as an angle on a color wheel ($0^\circ - 360^\circ$). In OpenCV, this is scaled to $0 - 180$.
- **Saturation (S):** The vividness or purity of the color.
- **Value (V):** The brightness or lightness of the pixel.

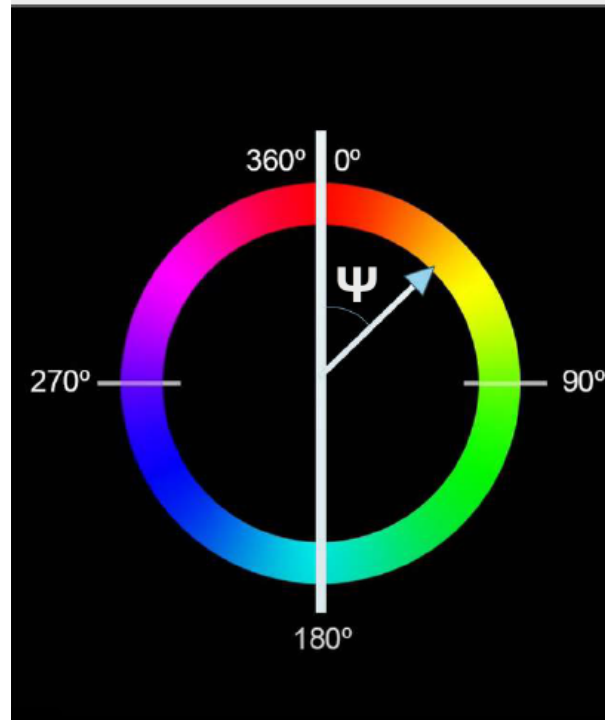


Figure 7: Visualizing Hue as an angle on the color wheel.

7.2 HSL vs. HSV

HSL stands for Hue, Saturation, and **L**ightness. The key difference is in how brightness is handled: in HSL, pure white has $L = 1$, while "true color" exists at $L = 0.5$.

8 Basic Image Enhancements

8.1 Brightness

Brightness refers to the overall lightness of an image. It is adjusted by adding a constant factor (β) to all pixel values.

$$I_{new} = I_{old} + \beta \quad (4)$$

This operation shifts the entire histogram to the right (brighter) or left (darker).

8.2 Contrast

Contrast is the difference in brightness between objects. It is adjusted by multiplying pixel values by a factor (α).

$$I_{new} = \alpha \times I_{old} \quad (5)$$

- $\alpha > 1$: Increases contrast (stretches the histogram).
- $\alpha < 1$: Decreases contrast (compresses the histogram).

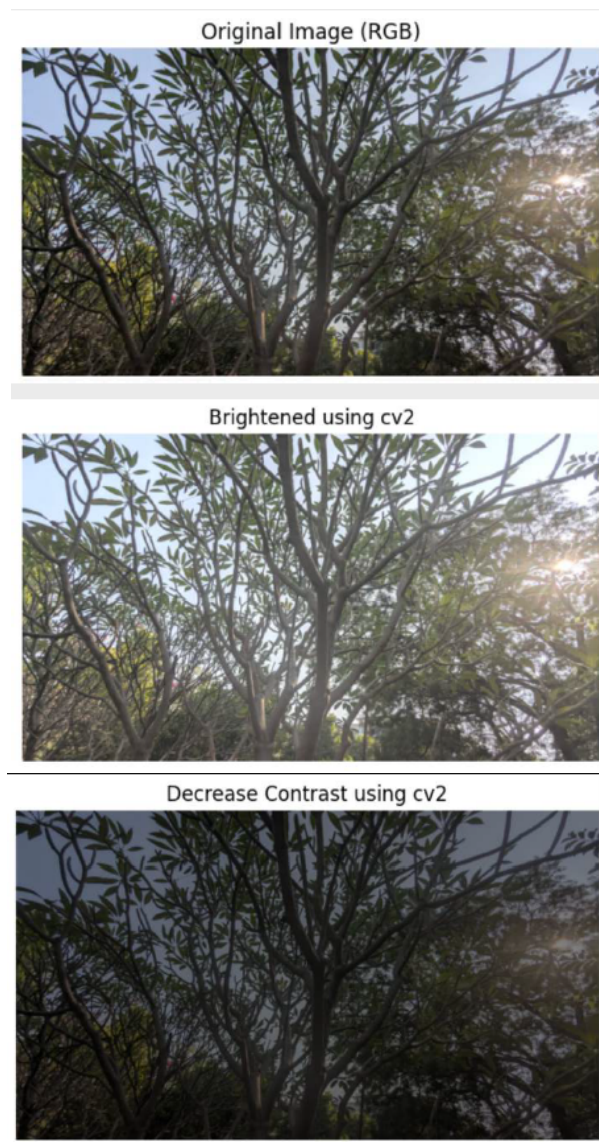


Figure 8: Comparison: Brightness Adjustment (Top) vs. Contrast Adjustment (Bottom).

9 Advanced Color Manipulation

9.1 Saturation and Vibrance

Saturation increases the intensity of all colors equally. However, this can lead to unnatural skin tones or "clipping" (loss of detail).

Vibrance is a "smart" saturation. It boosts low-saturation pixels while protecting already saturated pixels and skin tones.

$$S' = S + \alpha \cdot (1 - S) \cdot S \quad (6)$$

This results in a more natural, photographic look.



Figure 9: Result of applying Vibrance to boost dull colors without oversaturating.

9.2 Hue Shift

Because Hue is an angle, changing the color identity of an image involves rotating this angle. For example, a shift of $+120^\circ$ can turn Red pixels into Green pixels [?].



Figure 10: Effect of shifting Hue by 120° .

10 Gamma Correction and White Balance

10.1 Gamma Correction

Human eyes perceive brightness non-linearly; we are more sensitive to changes in dark tones. Gamma correction adjusts images to match this perception using a non-linear exponent (γ).

$$V_{out} = V_{in}^{(1/\gamma)} \quad (7)$$

A standard gamma value for displays is $\gamma = 2.2$.



Figure 11: Original Image vs. Gamma Corrected Image.

10.2 White Balance

White balance removes color casts (e.g., yellow indoor light) to make white objects appear truly white.

Gray World Assumption: This algorithm assumes the average color of a scene should be neutral gray. It calculates scale factors for each channel based on the average pixel intensities:

$$Scale = \frac{Avg_{Blue} + Avg_{Green} + Avg_{Red}}{3} \quad (8)$$

Each channel is then scaled to meet this average.

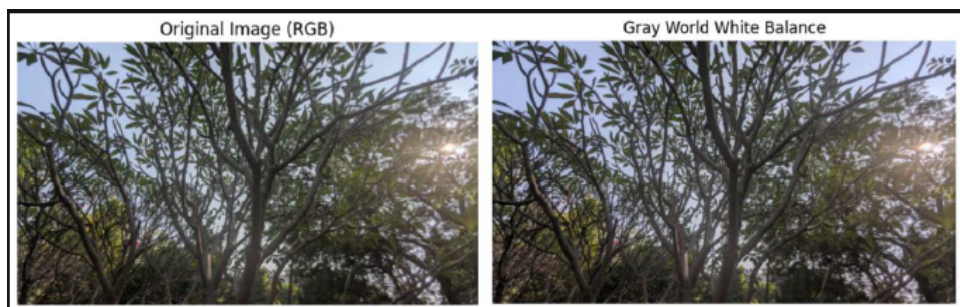


Figure 12: Original vs. White Balanced using Gray World Assumption.

Part III: Convolution and Filtering Operations

11 Convolution

Convolution is the fundamental operation in image processing for filtering, blurring, and edge detection. It involves sliding a "kernel" (a small matrix) over the image and computing the weighted sum of pixel neighbors.

For an image I and kernel K of size 3×3 :

$$(I * K)(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 I(x + i, y + j) \cdot K(i, j) \quad (9)$$

Kernel properties determine the effect:

- **Blur Kernels:** Sum of weights ≈ 1 .
- **Edge Detectors:** Sum of weights ≈ 0 .
- **Sharpen Kernels:** Sum of weights > 1 or < 0 .

12 Blurring Techniques

Blurring smooths an image by averaging pixel values with their neighbors.

12.1 Average Blur

All pixels in the kernel have equal weight.

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (10)$$

Effect: Reduces noise but creates a harsh, "soft" look that preserves no detail [?].

12.2 Gaussian Blur

Pixels closer to the center have higher weights, following a Gaussian distribution.

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad (11)$$

Effect: Provides a natural smoothing effect (similar to human vision focus) and preserves edges better than average blur [?].

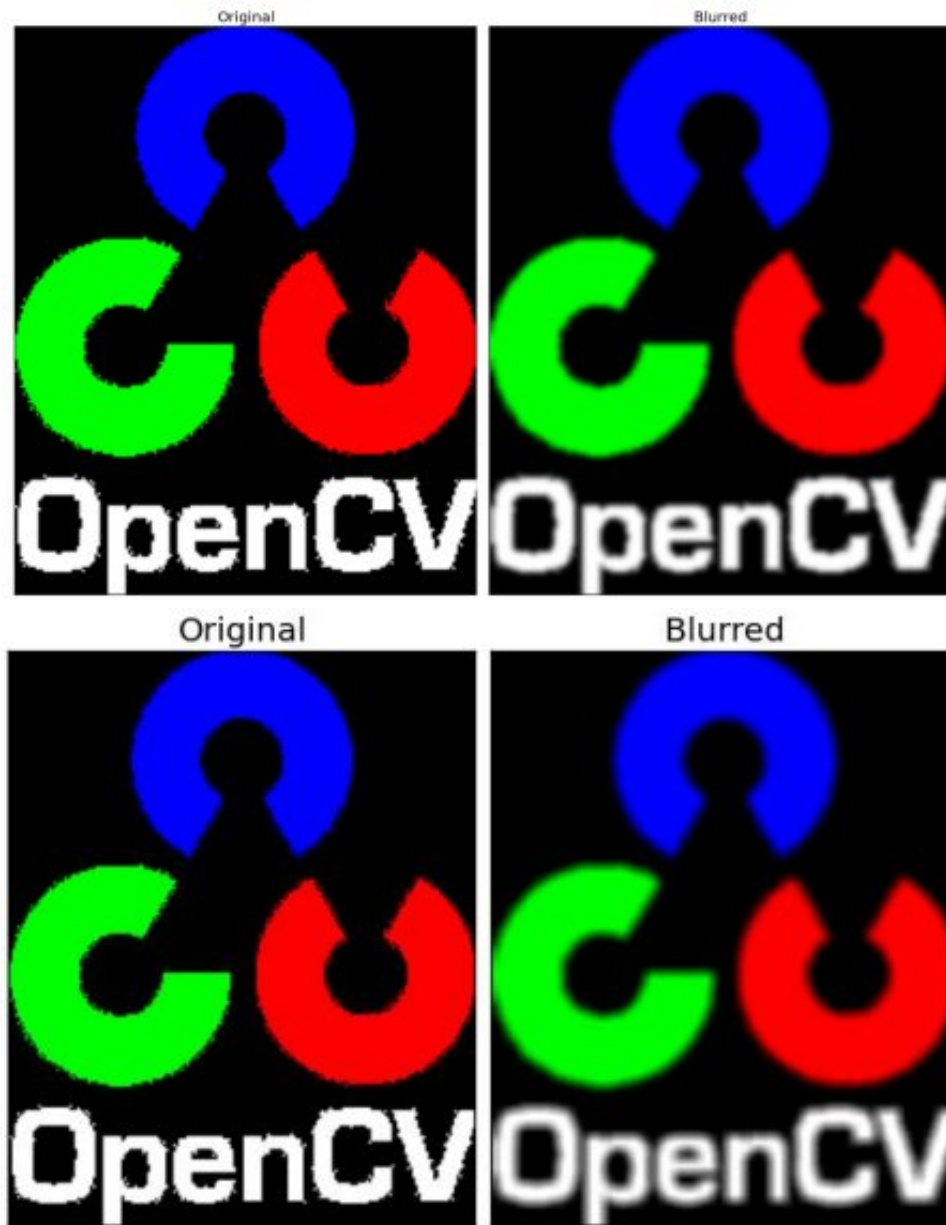


Figure 13: Visual comparison of Gaussian Blur(Bottom) and Average Blur(Top).

13 Gradients and Edge Detection

Gradients measure the directional change in pixel intensity. A large gradient indicates a strong edge.

13.1 Sobel Operator

The Sobel operator calculates gradients in the X and Y directions using specific kernels [?]:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (12)$$

- **Magnitude (Edge Strength):** $G = \sqrt{G_x^2 + G_y^2}$
- **Direction (Edge Orientation):** $\theta = \tan^{-1}(G_y/G_x)$

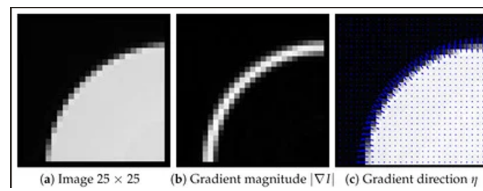


Figure 14: Gradient vector representing intensity change direction.

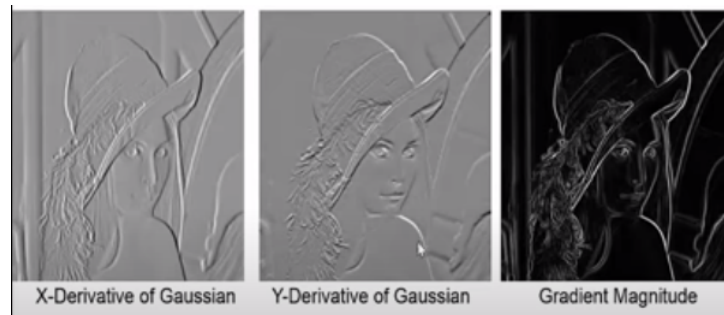


Figure 15: Result of Sobel Edge Detection showing gradient magnitude.

13.2 Canny Edge Detection

The Canny algorithm is the "gold standard" for edge detection, involving a multi-step process:

1. **Gaussian Blur:** To remove noise.
2. **Sobel Gradients:** To find intensity changes.
3. **Non-max Suppression:** To thin out edges.
4. **Double Threshold:** To identify strong vs. weak edges.
5. **Hysteresis:** To track edges by connecting weak edges to strong ones.

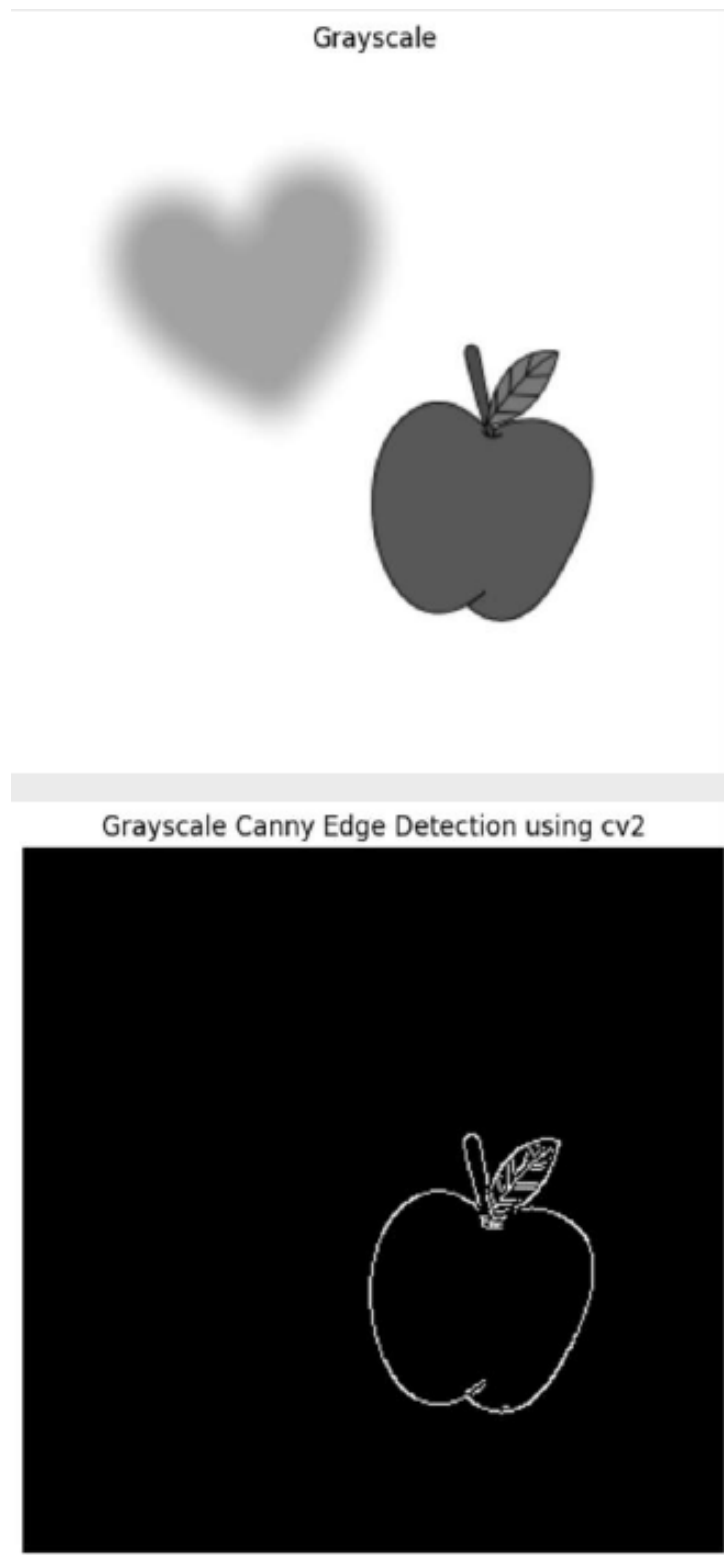


Figure 16: Result of Canny Edge Detection (clean, thin edges).

14 Sharpening

Sharpening highlights fine details and enhances edges.

14.1 Laplacian Sharpening

The Laplacian is a second-order derivative operator that detects regions of rapid intensity change.

$$\text{Laplacian Kernel} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (13)$$

To sharpen, we subtract the Laplacian from the original image:

$$I_{\text{sharp}} = I_{\text{original}} - \text{Laplacian}(I) \quad (14)$$

This amplifies the edges and fine textures.

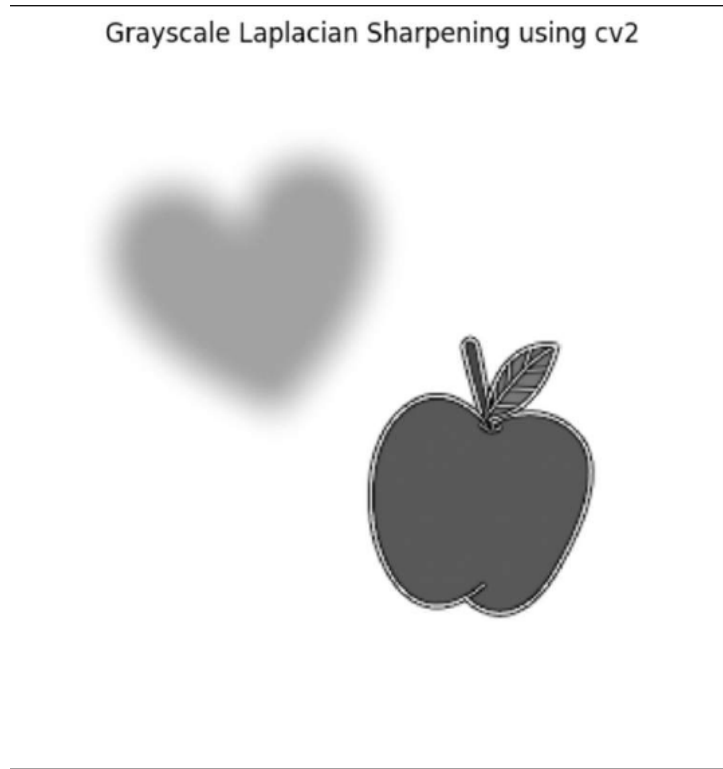


Figure 17: Laplacian Sharpening result.

14.2 Unsharp Masking

A technique derived from traditional photography:

1. Create a blurred version of the image (B).
2. Subtract the blur from the original to create a "mask" ($M = I - B$).
3. Add the mask back to the original: $I_{\text{sharp}} = I + kM$.

Here, k controls the amount of sharpness.

Grayscale Unsharp masking using cv2



Grayscale Unsharp masking using cv2



Figure 18: Result of Unsharp Masking.

15 Conclusion on Frequency Domain Connection

Convolution in the spatial domain is equivalent to multiplication in the frequency domain.

- **Gaussian Kernel** $\approx$ Low Pass Filter (attenuates high frequencies).
- **Sharpening Kernel** $\approx$ High Pass Filter (amplifies high frequencies).

Part IV: Feature Detection and Hough Transforms

16 Corner Detection

To detect features, we analyze how image intensity changes within a small window (patch) when shifted.

- **Flat Region:** Intensity remains approximately constant in all directions.
- **Edge Region:** Intensity changes significantly in only one direction (perpendicular to the edge).
- **Corner Region:** Intensity changes significantly in all directions.

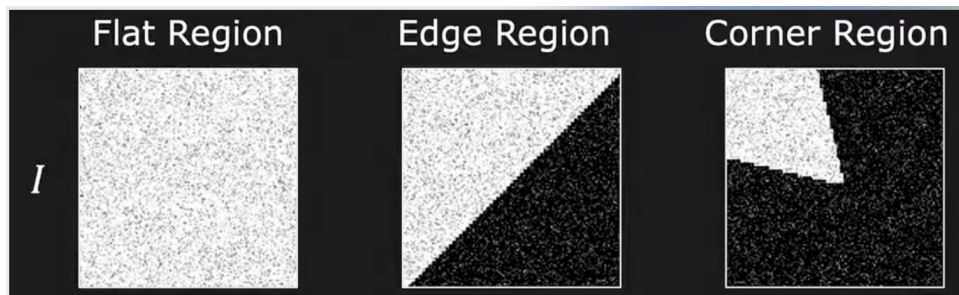


Figure 19: Visualizing Flat, Edge, and Corner regions based on intensity change.

17 Hough Transform for Line Detection

Unlike local edge detectors (Sobel, Canny), the Hough Transform is a global method that finds shapes by letting edge points "vote" in a parameter space.

17.1 The Duality Principle

There is a duality between the Image Space (x, y) and the Parameter Space (m, c) :

- A line in Image Space corresponds to a point in Parameter Space.
- A point in Image Space corresponds to a line in Parameter Space ($c = -xm + y$) [?].

Therefore, collinear points in the image will produce lines in the parameter space that all intersect at a single point (m, c) , which represents the line equation.

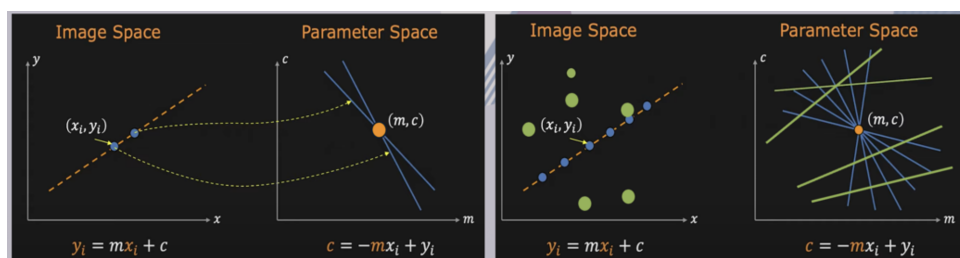


Figure 20: Duality: Points in Image Space map to Lines in Parameter Space.

17.2 Polar Representation

Using Cartesian parameters (m, c) is problematic because vertical lines have infinite slope ($m \rightarrow \infty$). To solve this, we use Polar Coordinates:

$$x \sin \theta - y \cos \theta + \rho = 0 \quad (15)$$

(Note: Standard convention is often $x \cos \theta + y \sin \theta = \rho$, but the concept remains the same: ρ is the perpendicular distance, and θ is the angle). In this space, a point in the image maps to a sinusoid in the (ρ, θ) accumulator.

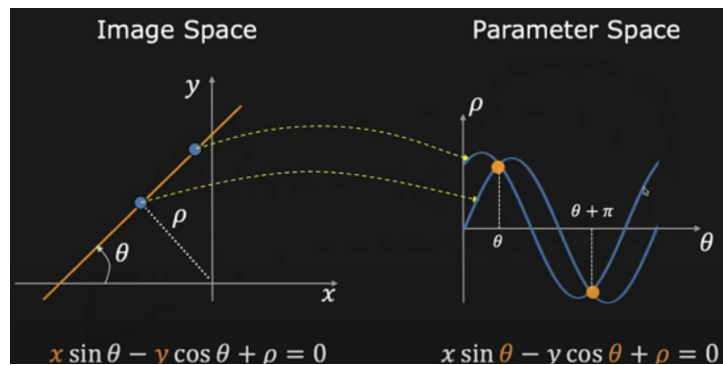


Figure 21: Mapping from Image Space to Sinusoids in Polar Hough Space.

17.3 Algorithm

1. Quantize the parameter space (ρ, θ) into an accumulator array.
2. Initialize all cells to zero.
3. For every edge point (x_i, y_i) , calculate ρ for all θ and increment the corresponding accumulator cell (Voting).
4. Local maxima in the accumulator correspond to strong lines in the image.

18 Hough Transform for Circles

18.1 Fixed Radius

If the radius r is known, the parameter space is 2D: (a, b) .

$$(x - a)^2 + (y - b)^2 = r^2 \quad (16)$$

Each edge point votes for a circle of radius r in the parameter space. The intersection of these circles identifies the center (a, b) .

18.2 Unknown Radius

If r is unknown, the parameter space becomes 3D: (a, b, r) , which is computationally expensive. **Optimization:** Use gradient direction information. The center of the circle must lie along the gradient vector at the edge point, reducing the search space.

- Circle: center (a, b) and radius r

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

- For an unknown radius r , unknown gradient direction

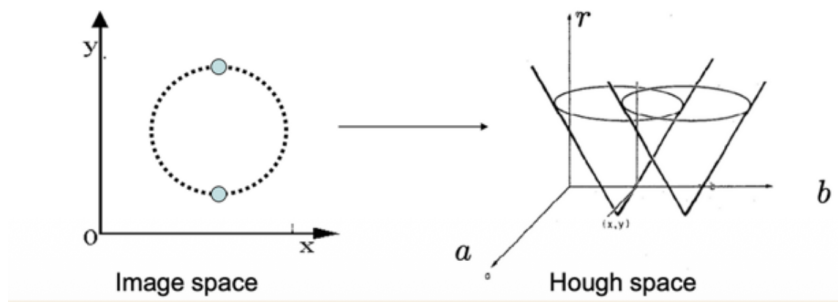


Figure 22: Circle detection via voting in parameter space.

19 Generalized Hough Transform (GHT)

The GHT extends the method to detect arbitrary shapes that have no simple mathematical equation.

19.1 R-Table and Voting

1. **Offline Phase:** Construct an R-Table. For a reference shape, store displacement vectors $(\vec{r})$ from boundary points to a reference center (x_c, y_c) , indexed by the gradient orientation ϕ . 2. **Online Phase:** For each edge pixel in the target image:

- Calculate gradient orientation.
- Look up corresponding vectors in the R-Table.
- Vote for potential centers in the accumulator array.

19.2 Scale and Rotation

To handle changes in scale (s) and rotation (θ), the parameter space must expand to 4 dimensions (x_c, y_c, s, θ) , making it computationally intensive:

$$x_c = x_i \pm s \cdot r \cos(\alpha + \theta) \quad (17)$$

$$y_c = y_i \pm s \cdot r \sin(\alpha + \theta) \quad (18)$$

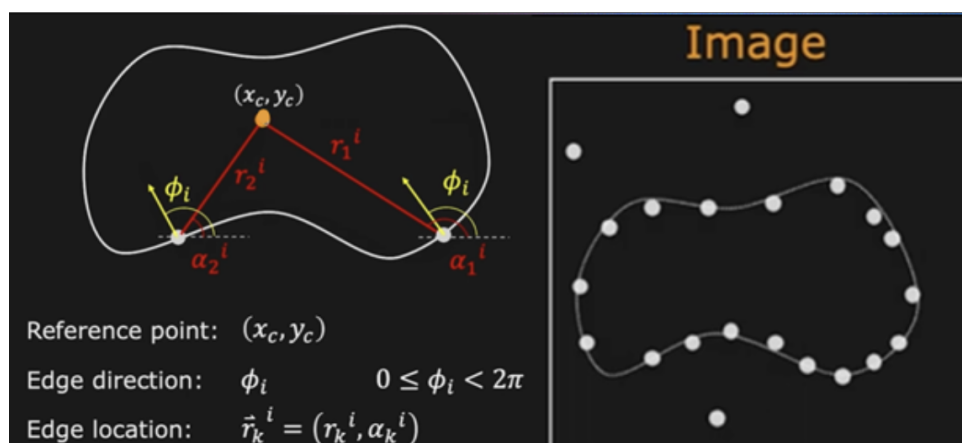


Figure 23: Generalized Hough Transform for arbitrary shape detection.

Part V: Feature Extraction and Object Classification

20 Color Features

Color is a primary discriminator for fruits. We analyze the Hue, Saturation, and Value (HSV) channels to extract statistical features.

20.1 Mean and Standard Deviation

For each channel (H, S, V), we calculate:

- **Mean Hue:** Determines the dominant color identity (e.g., Banana $\approx$ Yellow).
- **Std Dev Hue (σ_h):** Measures color uniformity. A fresh banana has low σ_h , while a spotted one has high σ_h .
- **Mean Saturation:** Indicates purity. Fresh fruits typically have high saturation.
- **Std Dev Value (σ_v):** Represents the spread of shadows and highlights (rough vs. smooth surfaces).

21 Shape Features

Shape features help distinguish fruits with similar colors (e.g., Green Apple vs. Pear). We use contours to extract these metrics.

21.1 Contour Properties

- **Area (A):** Total number of pixels inside the contour.
- **Perimeter (P):** Length of the contour boundary.
- **Compactness:** A measure of how circular an object is, defined as:

$$C = \frac{P^2}{A} \quad (19)$$

(Note: A perfect circle has minimum compactness).

- **Aspect Ratio:** Ratio of width to height of the bounding box.

21.2 Hu Moments

Hu Moments are a set of 7 numbers calculated using central moments that are invariant to:

- Translation
- Scale
- Rotation

These are crucial for recognizing the shape of the fruit regardless of its orientation or size in the image.

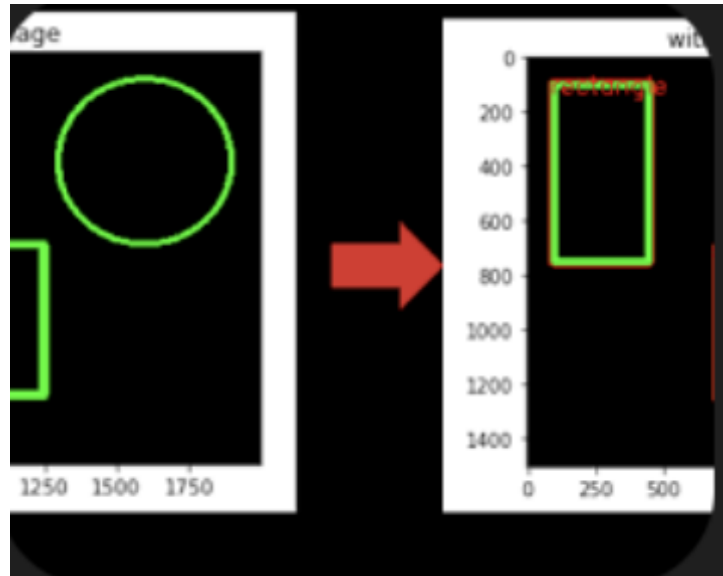


Figure 24: Contour detection used to extract shape metrics.

22 Texture Analysis: Local Binary Patterns (LBP)

Texture helps distinguish between smooth surfaces (e.g., Apple) and rough surfaces (e.g., Pineapple). LBP is a powerful texture descriptor.

22.1 LBP Algorithm

For every pixel in a grayscale image: 1. Select a 3×3 window around the center pixel. 2. Compare each of the 8 neighbors to the center pixel value. 3. If Neighbor $\geq$ Center, write 1. Otherwise, write 0. 4. Construct an 8-bit binary number (byte) from these comparison results (clockwise or counter-clockwise). 5. Convert this binary number to a decimal value (0-255) and replace the center pixel.

$$\text{LBP} = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p \quad s(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Figure 25: Calculating the LBP value for a 3×3 neighborhood.

The result is an LBP image where edges and textures are highlighted. A histogram of this LBP image serves as a feature vector for classification.

23 Implementation Strategy

The final implementation involves creating a custom Data Loader for the dataset.

- **Inputs:** Raw Images.
- **Processing:** Generate LBP image and Canny Edge image.
- **Features:** Extract 6 Color features + 6 Shape features (Linear features).
- **Output:** Class Label (Fruit Name).

This combination of deep features (from images) and handcrafted features (linear data) aims to maximize classification accuracy.

Part VI: Introduction to Machine Learning

24 Transition from Vision to Learning

Up to this point, our focus has been on **Computer Vision (CV)**—processing images as matrices, extracting features (edges, shapes, colors), and transforming signals (FFTs).

We now transition into **Machine Learning (ML)**. While CV focuses on "how to see," ML focuses on "how to learn" patterns from data to make predictions. We move from deterministic algorithms (like Canny Edge Detection) to probabilistic models that learn parameters from data [1].

25 Types of Learning

25.1 Supervised Learning

In supervised learning, the model is provided with a dataset containing both inputs (X) and correct outputs/labels (Y). The goal is to learn a mapping function.

- **Regression:** The output Y is a continuous value (e.g., predicting house prices).
- **Classification:** The output Y is a discrete class label (e.g., "Cat" vs. "Dog").

25.2 Unsupervised Learning

In unsupervised learning, we are given only the input data (X) without labels. The goal is to discover hidden structures or patterns.

- **Clustering:** Grouping similar data points (e.g., K-Means).
- **Dimensionality Reduction:** Simplifying data while retaining information (e.g., PCA).

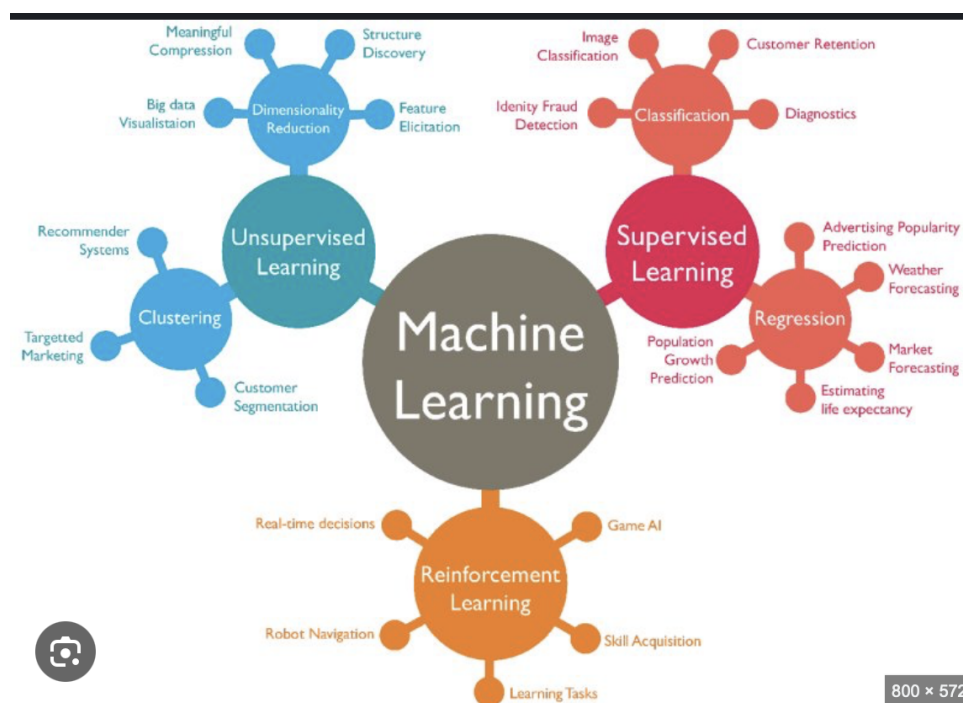


Figure 26: Overview of Supervised vs. Unsupervised Learning.

26 Probabilistic Foundations of Regression

Linear Regression is often introduced as "fitting a line that minimizes error." However, mathematically, it is derived from the properties of the **Normal (Gaussian) Distribution**.

26.1 The Normal Distribution

Data samples in nature often follow a bell curve distribution defined by:

$$N(x|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \quad (20)$$

Where μ is the mean and σ is the standard deviation.

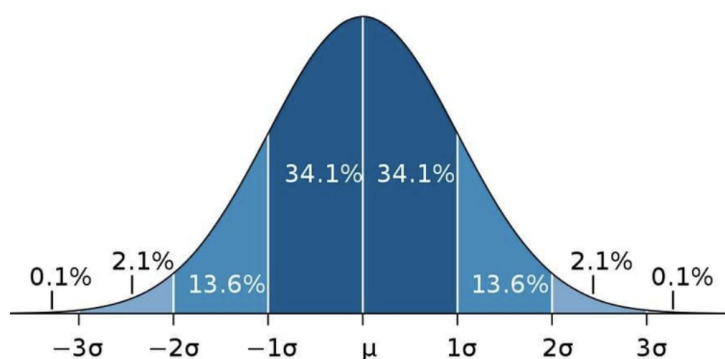


Figure 27: The Normal Distribution: The mathematical basis for MSE.

26.2 MLE vs. MSE

Why do we use Mean Squared Error (MSE) in regression? It is not arbitrary; it comes from **Maximum Likelihood Estimation (MLE)**.

1. We assume the target value y is the result of a deterministic function of x plus some random Gaussian noise:

$$y = W^T x + \epsilon, \quad \text{where } \epsilon \sim N(0, \sigma) \quad (21)$$

2. This implies that y itself follows a normal distribution centered at the prediction $W^T x$:

$$P(y|x, W) \sim N(W^T x, \sigma) \quad (22)$$

3. To find the best weights W , we maximize the **Likelihood** of observing our data (making the data "most probable"). Maximizing the likelihood corresponds to maximizing the exponent term:

$$\text{Maximize } \prod e^{-\frac{1}{2}(y_i - \hat{y}_i)^2} \quad (23)$$

4. Maximizing this negative exponent is mathematically equivalent to **minimizing** the term inside:

$$\text{Minimize } \sum (y_i - \hat{y}_i)^2 \quad (24)$$

Conclusion: Minimizing the Squared Error (MSE) is statistically equivalent to maximizing the likelihood of the data under the assumption of Gaussian noise.

27 Linearity in Regression

A common misconception is that "Linear Regression" can only model straight lines.

27.1 What does "Linear" mean?

Linear regression is linear with respect to the **parameters (weights)**, not necessarily the input features.

$$y = w_0 + w_1x + w_2x^2 + w_3x^3 \quad (25)$$

This is a polynomial equation, producing a curve. However, it is still a **Linear Regression** model because it is a linear combination of the weights ($w_0, w_1 \dots$).

27.2 Basis Expansion

We can model non-linear data relationships by transforming the input features into a higher-dimensional space (e.g., $[1, x, x^2, x^3]$) and performing linear regression in that feature space.

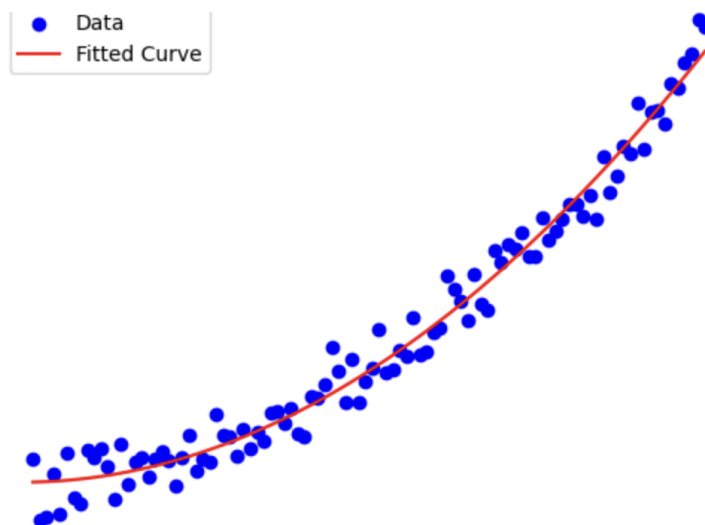


Figure 28: Fitting non-linear data using Linear Regression on polynomial features.

Part VII: Introduction to Neural Networks

28 Model Complexity: Underfitting vs. Overfitting

When training machine learning models, finding the right balance in model complexity is crucial.

28.1 Underfitting (High Bias)

This occurs when the model is too simple to capture the underlying trend of the data. For example, trying to fit a curve with a straight line.

$$f(x) = w_1x + w_0 \quad (26)$$

The model has high bias because it makes strong assumptions about the data shape (linearity) that are incorrect.

28.2 Overfitting (High Variance)

This occurs when the model is too complex and learns the noise in the training data rather than the actual signal. It might pass through every data point but fails to generalize to new data.

$$f(x) = w_nx^n + w_{n-1}x^{n-1} + \dots + w_0 \quad (27)$$

Using a polynomial of extremely high degree (n) creates a "wiggly" function that fits training data perfectly but fluctuates wildly.

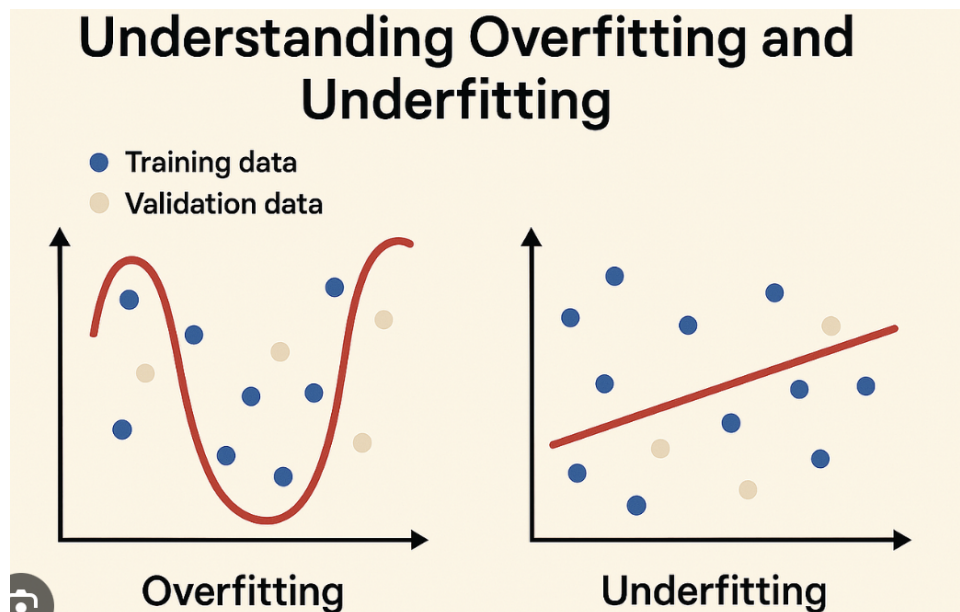


Figure 29: Visualizing the trade-off between Underfitting (Left) and Overfitting (Right).

29 Multi-Layer Perceptrons (MLP)

To solve complex, non-linear problems, we move from simple regression to **Neural Networks**. An MLP consists of neurons organized into layers: an Input Layer, one or more Hidden Layers, and an Output Layer.

29.1 The Need for Non-Linearity

A common question is: "Why not just stack linear layers?" If we stack linear transformations, the result is still linear:

$$y = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W_{new}x + b_{new} \quad (28)$$

To learn complex boundaries, we must introduce a **Non-Linear Activation Function** (σ) after each linear step [?].

30 Activation Functions: ReLU

The Rectified Linear Unit (ReLU) is one of the most popular activation functions in deep learning.

30.1 Mathematical Definition

ReLU is a piecewise linear function that outputs the input directly if it is positive, otherwise, it outputs zero:

$$\text{ReLU}(x) = \max(0, x) \quad (29)$$

Despite being piecewise linear, it introduces the necessary non-linearity to the network.

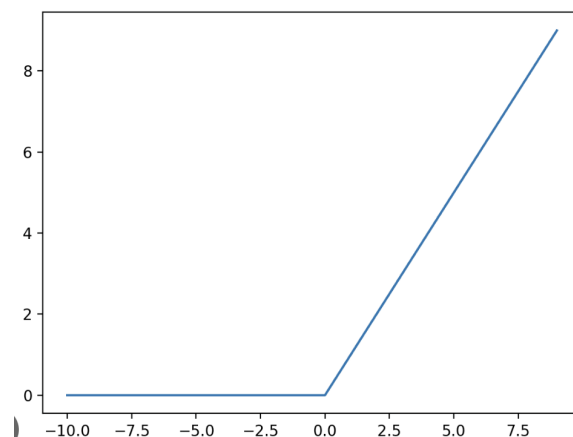


Figure 30: The ReLU activation function.

31 Universal Approximation Theorem

This theorem states that a feed-forward network with a single hidden layer containing a finite number of neurons can approximate continuous functions on compact subsets of R^n .

31.1 Intuition with ReLU

We can combine multiple ReLU units, each with different weights (slopes) and biases (shifts), to construct complex shapes. By adding these "kinks" together, we can approximate any curve:

$$y = K_1 \text{ReLU}(w_1x + b_1) + K_2 \text{ReLU}(w_2x + b_2) + \dots + K_n \text{ReLU}(w_nx + b_n) \quad (30)$$

Each neuron contributes a "piece" to the final function, allowing the network to fit complex data distributions.

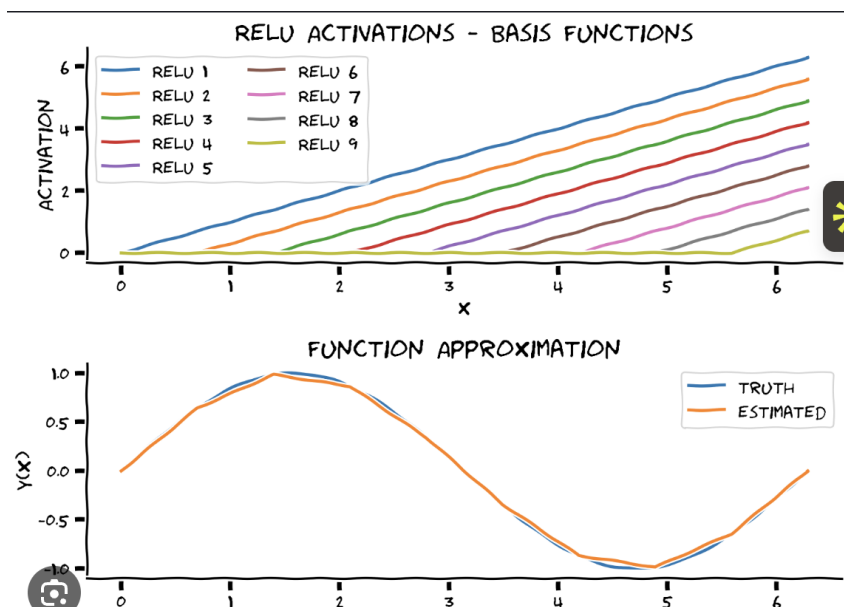


Figure 31: Constructing a complex function by summing multiple ReLU outputs.

32 Deep Network Architecture

In a deep network, the output of one layer (f_{l-1}) becomes the input to the next layer (f_l). For a network with weights W and biases b :

$$f_l = \text{ReLU}(W_l \cdot f_{l-1} + b_l) \quad (31)$$

This hierarchical structure allows the network to learn features at different levels of abstraction.

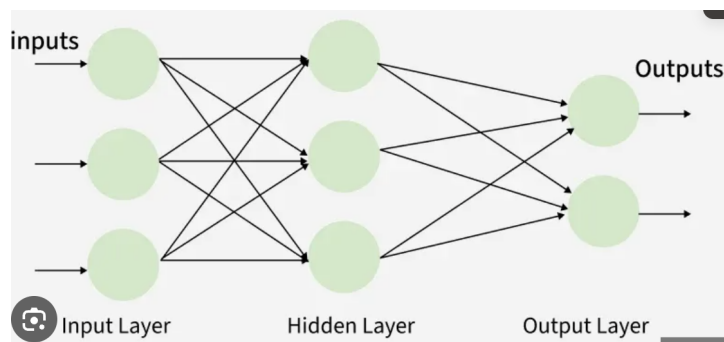


Figure 32: Structure of a Multi-Layer Perceptron.

Part VIII: Convolutional Neural Networks (CNNs)

33 From MLPs to CNNs

While Multi-Layer Perceptrons (MLPs) are powerful universal approximators, they face significant challenges when processing images.

33.1 The Problem with MLPs for Images

In an MLP, every input pixel is connected to every neuron in the next layer (Dense connection).

- **Parameter Explosion:** For a small image of 28×28 , a single neuron has 784 weights. For a high-resolution image (1000×1000), a single neuron would need 1,000,000 weights. This leads to massive computational costs and overfitting.
- **Loss of Spatial Structure:** Flattening an image into a 1D vector destroys the spatial relationships (neighboring pixels are correlated).
- **Lack of Translational Invariance:** An MLP learns that a "cat" in the top-left corner is different from a "cat" in the bottom-right corner. It has to relearn the same features for every position.

33.2 The CNN Solution

Convolutional Neural Networks (CNNs) solve these issues by using **Kernels (Filters)** instead of dense weights.

- **Parameter Efficiency:** A kernel (e.g., 3×3) has only 9 parameters, regardless of the image size.
- **Translational Invariance:** The same kernel slides across the entire image, detecting the same feature (e.g., an edge) wherever it appears.

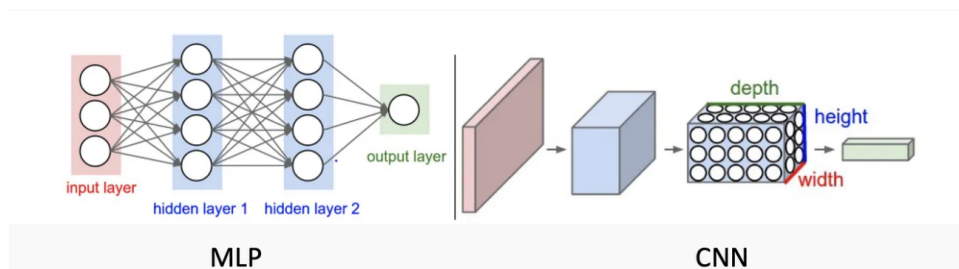


Figure 33: Difference between MLP (Dense) and CNN (Shared Weights).

34 Key CNN Operations

34.1 Convolutional Layers

The core operation is the convolution (technically cross-correlation) of an input image I with a learnable filter K :

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j) \quad (32)$$

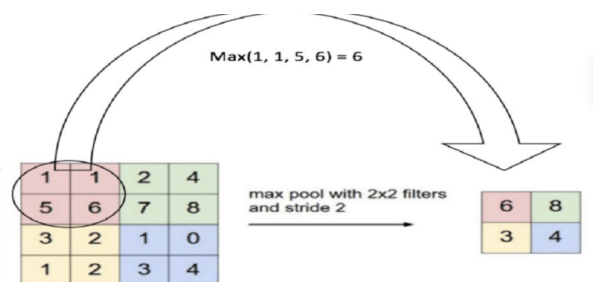
The network learns the values of K during training to extract relevant features (edges, textures, patterns).

34.2 Max Pooling

Pooling layers are used to reduce the spatial dimensions (height and width) of the feature maps, reducing computation and making the model more robust to small shifts. **Max Pooling** selects the maximum value from a local window (e.g., 2×2):

$$y_{i,j} = \max_{(p,q) \in R_{i,j}} x_{p,q} \quad (33)$$

This keeps the most prominent feature in that region while discarding less important details.

Figure 34: Max Pooling operation reducing a 4×4 grid to 2×2 .

35 Model Optimization: Pruning

As neural networks grow deeper, they consume significant memory and compute power. **Pruning** is a technique to compress models for efficient deployment.

35.1 Magnitude-Based Pruning

We observe that many weights in a trained network are very close to zero. These weights contribute little to the output.

- **Strategy:** Set a threshold (e.g., the 80th percentile of weight magnitudes).
- **Action:** Zero out all weights $|w| < \text{threshold}$.

- **Result:** A sparse matrix that requires significantly less storage and computation, often preserving 95%+ of the original accuracy.

35.2 Quantization

Another optimization is converting weights from high-precision 'float32' (32-bit) to lower-precision formats like 'int8' (8-bit). This reduces model size by $4\times$ and speeds up inference on hardware.

Part IX: Deep Learning with PyTorch

36 Why PyTorch?

We now transition from theoretical concepts to practical implementation using *PyTorch*. While TensorFlow was historically significant, PyTorch has become the dominant framework for research and production (used in $\sim 92\%$ of recent papers).

36.1 PyTorch vs. TensorFlow

- **Dynamic Computation Graphs:** PyTorch builds the graph on-the-fly (Python-first), making debugging easier (standard Python tools work).
- **Research Friendly:** It allows for rapid prototyping and is the default for Generative AI (LLMs, Diffusers).

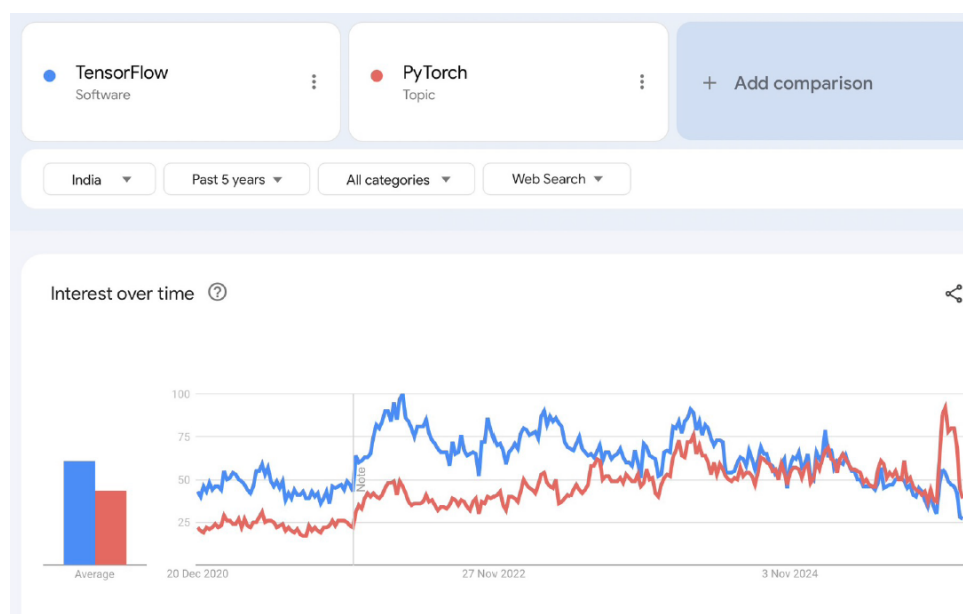


Figure 35: Adoption trends: PyTorch vs. TensorFlow.

37 The Fundamental Unit: Tensors

A **Tensor** is the core data structure in PyTorch. It is mathematically similar to a NumPy array but comes with two critical superpowers:

1. **GPU Acceleration:** Tensors can be moved to the GPU (using `.to('cuda')`) for massively parallel computation.
2. **Automatic Differentiation:** Tensors track the history of operations applied to them to compute gradients automatically.

$$\text{Tensor} \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n} \quad (34)$$

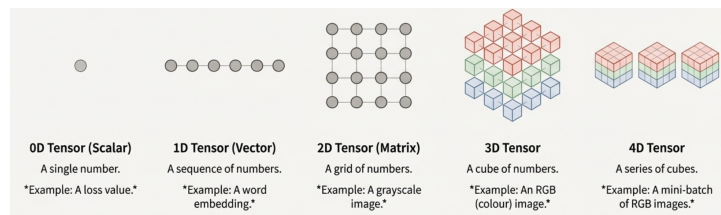


Figure 36: Visualizing Tensors: From Scalars (0D) to n-Dimensional structures.

38 The Engine: Autograd

Training neural networks requires calculating gradients ($\frac{\partial L}{\partial w}$) for backpropagation. Doing this manually is error-prone and tedious.

38.1 Computational Graphs

PyTorch uses **Autograd** to build a Directed Acyclic Graph (DAG) of operations.

- **Nodes:** Tensors (Data).
- **Edges:** Functions (Operations like $+$, $*$, $\sin$).

When you execute a forward pass, PyTorch builds this graph dynamically. When you call `backward()`, it traverses the graph in reverse to calculate gradients using the Chain Rule.

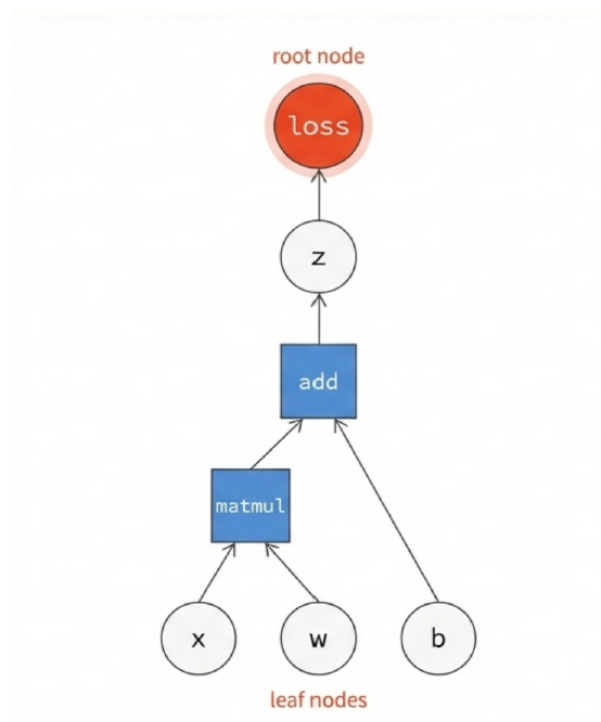


Figure 37: A dynamic computation graph (DAG) tracking operations.

38.2 Controlling Gradients

- `requires_grad=True`: Tells PyTorch to track operations on this tensor (essential for weights and biases).

- `torch.no_grad()`: A context manager that pauses the gradient engine. This is used during **Inference** to save memory and computation.

39 The PyTorch Workflow

The standard pipeline for training a model in PyTorch consists of five key steps:

1. **Data Preparation:** Using `Dataset` and `DataLoader` to batch and shuffle inputs.
2. **Forward Pass:** Passing input data through the model to get predictions ($\hat{y}$).
3. **Loss Calculation:** Computing the error between predictions and ground truth ($L(\hat{y}, y)$).
4. **Backward Pass:** Calling `loss.backward()` to compute gradients.
5. **Optimizer Step:** Updating the model parameters using the computed gradients (`optimizer.step()`).

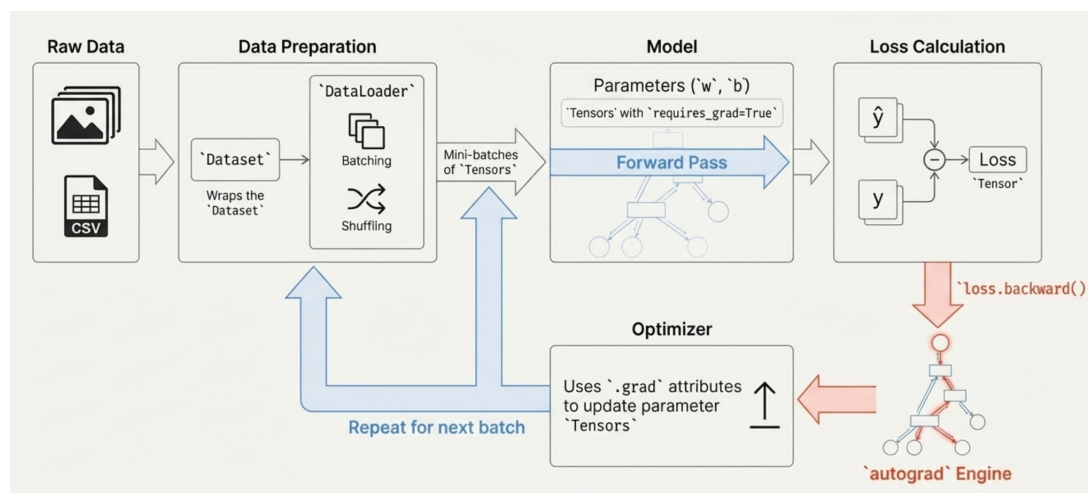


Figure 38: The complete PyTorch training workflow.

References

- [1] CV Session 1
- [2] CV Session 2
- [3] CV Session 3
- [4] CV Session 4
- [5] CV Session 5
- [6] ML Session 1
- [7] ML Session 2
- [8] ML Session 3
- [9] PyTorch Lecture